

Bayesian Z-residual Diagnostics for Detecting Nonlinearity in Inverse Gaussian regression (Section 3.2)

Anonymous Author

2026-07-06

1 Installing and Loading Required Packages

Install Zresidual

```
# During package development, run this once from the package root before rendering:
# renv::install(here::here(), dependencies = TRUE)
#
# For a clean external environment, install the released or review version instead:
# if (!requireNamespace("remotes", quietly = TRUE)) install.packages("remotes")
# remotes::install_github("anonymous/Zresidual", upgrade = "never", dependencies = TRUE)
```

Loading required packages

```
pkgs <- c("brms", "dplyr", "tibble", "gt", "Zresidual")
missing_pkgs <- pkgs[!vapply(pkgs, requireNamespace, logical(1), quietly = TRUE)]

if (length(missing_pkgs)) {
  stop(
    "Please install the following packages before rendering this vignette: ",
    paste(missing_pkgs, collapse = ", "),
    call. = FALSE
  )
}
```

```
invisible(lapply(pkgs, function(p) {
  suppressPackageStartupMessages(library(p, character.only = TRUE))
}))

brms_backend <- "rstan"
if (requireNamespace("cmdstanr", quietly = TRUE)) {
  cmdstan_path <- try(cmdstanr::cmdstan_path(), silent = TRUE)
  if (!inherits(cmdstan_path, "try-error") && nzchar(cmdstan_path) && dir.exists(cmdstan_path)) {
    brms_backend <- "cmdstanr"
  }
}

nc <- parallel::detectCores(logical = FALSE)
if (!is.na(nc) && nc > 1) options(mc.cores = nc - 1)
brms_cores <- min(2, getOption("mc.cores", 1))
```

2 Introduction

This vignette demonstrates how Z-residual diagnostics can be used for a continuous Bayesian regression model. We use an inverse Gaussian (IG) example because, for continuous responses, the randomized survival probability (RSP) reduces to the ordinary survival probability and no auxiliary randomization is needed.

The demo uses one representative dataset with $n = 50$. It fits two Bayesian IG models with the same response family but different mean structures: a flexible piecewise model and a simpler linear model. We then compute posterior and ISCV Z-residual diagnostics, together with comparable PPC and HPC checks. The emphasis is on the applied workflow: compute residuals, inspect residual plots, apply targeted tests against covariates, and compare the conclusions with PPC/HPC summaries.

3 Data-Generating Model

For each observation, the covariates are generated as

$$X_{i,1} \sim \text{Unif}(0.5, 15.5), \quad X_{i,2} \sim \text{Binom}(1, 0.5), \quad X_{i,3} \sim \text{Binom}(1, 0.5),$$

and the five independent non-primary predictors are

$$Z_{i,1}, \dots, Z_{i,5} \sim N(0, 1).$$

Let $c = 7.95$ and define

$$B_i = (X_{i,1} - c)^2 I(X_{i,1} < c), \quad A_i = (X_{i,1} - c)^2 I(X_{i,1} \geq c).$$

The response is generated from an inverse Gaussian distribution,

$$Y_i \sim \text{IG}(\mu_i, \lambda), \quad \lambda = 25,$$

where

$$\log(\mu_i) = -0.80 + 0.012B_i + 0.006A_i + 0.40X_{i,2} - 0.25X_{i,3} + 0.12Z_{i,1} - 0.10Z_{i,2} + 0.08Z_{i,3} - 0.06Z_{i,4} + 0.05Z_{i,5}.$$

The two fitted Bayesian IG models use the same response family and log link. They differ only in the mean structure:

$$\text{Piecewise IG: } \log(\mu_i) = \alpha_0 + \alpha_1 B_i + \alpha_2 A_i + \alpha_3 X_{i,2} + \alpha_4 X_{i,3} + \sum_{j=1}^5 \gamma_j Z_{i,j},$$

$$\text{Linear IG: } \log(\mu_i) = \alpha_0 + \alpha_1 X_{i,1} + \alpha_2 X_{i,2} + \alpha_3 X_{i,3} + \sum_{j=1}^5 \gamma_j Z_{i,j}.$$

```
# Michael-Schucany-Haas generator for IG(mu, lambda).
# Under this parameterization, E(Y) = mu and Var(Y) = mu^3 / lambda.
rinvgauss_shape <- function(n, mu, shape) {
  if (length(mu) == 1L) mu <- rep(mu, n)
  if (length(shape) == 1L) shape <- rep(shape, n)
  stopifnot(length(mu) == n, length(shape) == n)

  v <- stats::rnorm(n)^2
  x <- mu + (mu^2 * v) / (2 * shape) -
    (mu / (2 * shape)) * sqrt(4 * mu * shape * v + mu^2 * v^2)
  u <- stats::runif(n)
  ifelse(u <= mu / (mu + x), x, mu^2 / x)
}
```

```

sim_piecewise_ig <- function(sample_size = 50, shape = 25, cutpoint = 7.95, seed = 1) {
  set.seed(seed)
  n <- sample_size

  x1 <- stats::runif(n, min = 0.5, max = 15.5)
  x2 <- stats::rbinom(n, size = 1, prob = 0.5)
  x3 <- stats::rbinom(n, size = 1, prob = 0.5)

  z1 <- stats::rnorm(n)
  z2 <- stats::rnorm(n)
  z3 <- stats::rnorm(n)
  z4 <- stats::rnorm(n)
  z5 <- stats::rnorm(n)

  below <- (x1 - cutpoint)^2 * as.numeric(x1 < cutpoint)
  above <- (x1 - cutpoint)^2 * as.numeric(x1 >= cutpoint)

  eta_true <- -0.80 +
    0.012 * below +
    0.006 * above +
    0.40 * x2 -
    0.25 * x3 +
    0.12 * z1 -
    0.10 * z2 +
    0.08 * z3 -
    0.06 * z4 +
    0.05 * z5

  mu_true <- exp(eta_true)
  y <- rinvgauss_shape(n = n, mu = mu_true, shape = shape)

  tibble::tibble(
    y = y,
    x1 = x1,
    x2 = x2,
    x3 = x3,
    z1 = z1,
    z2 = z2,

```

```

    z3 = z3,
    z4 = z4,
    z5 = z5,
    below = below,
    above = above,
    eta_true = eta_true,
    mu_true = mu_true
  )
}

fit_or_read_brms <- function(
  file,
  formula,
  data,
  seed,
  family,
  prior,
  chains = 4,
  iter = 2000,
  warmup = 1000
) {
  if (!force_refit && file.exists(file)) {
    return(readRDS(file))
  }

  fit <- brms::brm(
    formula = formula,
    family = family,
    prior = prior,
    data = data,
    backend = brms_backend,
    refresh = 0,
    cores = brms_cores,
    chains = chains,
    iter = iter,
    warmup = warmup,
    seed = seed,
    control = list(adapt_delta = 0.95, max_treedepth = 12),
    save_pars = brms::save_pars(all = TRUE)
  )
}

```

```

)

saveRDS(fit, file = file)
fit
}

call_zresidual_fun <- function(fun_name, ...) {
  fun <- try(getExportedValue("Zresidual", fun_name), silent = TRUE)
  if (inherits(fun, "try-error")) {
    fun <- get(fun_name, envir = asNamespace("Zresidual"))
  }
  fun(...)
}

as_pvalue <- function(x) {
  out <- suppressWarnings(try(as.numeric(x), silent = TRUE))
  if (inherits(out, "try-error") || length(out) == 0L) NA_real_ else out[1]
}

sw_pvalue <- function(zres) {
  out <- try(call_zresidual_fun("sw.test.zresid", zres), silent = TRUE)
  if (inherits(out, "try-error")) NA_real_ else as_pvalue(out)
}

aov_pvalue <- function(zres, zcov, x, k_anova = 10) {
  out <- try(
    call_zresidual_fun("aov.test.zresid", zres, X = x, zcov = zcov, k.anova = k_anova),
    silent = TRUE
  )
  if (inherits(out, "try-error")) {
    out <- try(
      call_zresidual_fun("aov.test.zresid", zres, X = x, zcov = zcov, k_anova = k_anova),
      silent = TRUE
    )
  }
  if (inherits(out, "try-error")) NA_real_ else as_pvalue(out)
}

bartlett_pvalue <- function(zres, zcov, x, k_anova = 10) {

```

```

out <- try(
  call_zresidual_fun("bartlett.test.zresid", zres, X = x, zcov = zcov, k.anova = k_anova),
  silent = TRUE
)
if (inherits(out, "try-error")) {
  out <- try(
    call_zresidual_fun("bartlett.test.zresid", zres, X = x, zcov = zcov, k_anova = k_anova),
    silent = TRUE
  )
}
if (inherits(out, "try-error")) NA_real_ else as_pvalue(out)
}

z_values <- function(zres, irep = 1) {
  if (is.matrix(zres) || is.data.frame(zres)) {
    return(as.numeric(zres[, min(irep, ncol(zres))]))
  }
  z_df <- suppressWarnings(try(as.data.frame(zres), silent = TRUE))
  if (!inherits(z_df, "try-error") && ncol(z_df) >= 1L) {
    num_cols <- vapply(z_df, is.numeric, logical(1))
    if (any(num_cols)) return(as.numeric(z_df[[which(num_cols)[1]]]))
  }
  as.numeric(zres)
}

chisq_z_pvalue <- function(zres, irep = 1) {
  out <- try(call_zresidual_fun("chisq.test.zresid", zres), silent = TRUE)
  if (!inherits(out, "try-error")) return(as_pvalue(out))

  z <- z_values(zres, irep = irep)
  z <- z[is.finite(z)]
  if (!length(z)) return(NA_real_)
  stats::pchisq(sum(z^2), df = length(z), lower.tail = FALSE)
}

safe_ppc <- function(...) {
  out <- try(call_zresidual_fun("ppc", ...), silent = TRUE)
  if (inherits(out, "try-error")) {
    warning("PPC failed: ", conditionMessage(attr(out, "condition")), call. = FALSE)
  }
}

```

```

    return(NULL)
  }
  out
}

safe_hpc <- function(...) {
  out <- try(call_zresidual_fun("hpc", ...), silent = TRUE)
  if (inherits(out, "try-error")) {
    warning("HPC failed: ", conditionMessage(attr(out, "condition")), call. = FALSE)
    return(NULL)
  }
  out
}

predcheck_scalar <- function(obj, field) {
  if (is.null(obj) || !is.list(obj) || is.null(obj[[field]])) return(NA_real_)
  suppressWarnings(as.numeric(obj[[field]])[1])
}

predcheck_aov <- function(obj, x_name) {
  if (is.null(obj) || !is.list(obj) || is.null(obj$aov_rpp_all)) return(NA_real_)
  target_name <- paste0("AOV[", x_name, "]")
  if (!target_name %in% names(obj$aov_rpp_all)) return(NA_real_)
  suppressWarnings(as.numeric(obj$aov_rpp_all[target_name])[1])
}

```

4 Simulate One Representative Dataset

To keep the rendered vignette fast and reproducible, we work with a single representative dataset with $n = 50$. The seed is fixed so that the same dataset is generated each time the demo is rendered.

```

sample_size <- 50
sim_id <- 75
seed_data <- 20260505 + sim_id - 1

ig_data_file <- file.path(
  resources_path,
  sprintf("piecewise_ig_data_n%03d_sim%03d.rds", sample_size, sim_id)
)

```



```

)

if (!force_rerun && file.exists(ig_data_file)) {
  dat_ig <- readRDS(ig_data_file)
} else {
  dat_ig <- sim_piecewise_ig(sample_size = sample_size, shape = 25, seed = seed_data)
  saveRDS(dat_ig, ig_data_file)
}

dplyr::glimpse(dat_ig)

```

Rows: 50

Columns: 13

```

$ y      <dbl> 0.4599244, 0.3120882, 0.6758396, 0.3745891, 0.5641120, 0.3184~
$ x1     <dbl> 9.900145, 7.352106, 2.768285, 6.911368, 13.956082, 10.562096,~
$ x2     <int> 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0~
$ x3     <int> 1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1~
$ z1     <dbl> -0.78496163, 0.26660848, 0.60690758, -1.08483962, 0.31914549,~
$ z2     <dbl> -1.48488183, 0.24610618, 2.51314042, 0.26737199, -0.16613788,~
$ z3     <dbl> 2.12206600, 0.50150961, -0.13201964, -0.09286781, -1.18772550~
$ z4     <dbl> -0.91735300, 0.64635332, 0.92730232, 0.85598452, -1.30382911,~
$ z5     <dbl> -0.003870632, -0.958915442, -0.788003845, -0.319131332, 0.235~
$ below  <dbl> 0.0000000, 0.3574772, 26.8501746, 1.0787555, 0.0000000, 0.000~
$ above  <dbl> 3.803064e+00, 0.000000e+00, 0.000000e+00, 0.000000e+00, 3.607~
$ eta_true <dbl> -0.74827590, -1.08493408, -0.36188294, -1.01871795, -0.533686~
$ mu_true <dbl> 0.4731817, 0.3379241, 0.6963639, 0.3610575, 0.5864389, 0.3275~

```

5 Fit the Piecewise and Linear Bayesian IG Models

```

ig_family <- brms::brmsfamily("inverse.gaussian", link = "log")

ig_prior <- c(
  brms::prior(normal(0, 1.5), class = "Intercept"),
  brms::prior(normal(0, 1.0), class = "b")
)

formula_piecewise <- brms::bf(y ~ below + above + x2 + x3 + z1 + z2 + z3 + z4 + z5)

```

```

formula_linear    <- brms::bf(y ~ x1 + x2 + x3 + z1 + z2 + z3 + z4 + z5)

fit_pieewise <- fit_or_read_brms(
  file = file.path(resources_path, sprintf("fit_pieewise_ig_n%03d_sim%03d.rds", sample_size, sim_id)),
  formula = formula_pieewise,
  data = dat_ig,
  seed = 20260506 + sim_id - 1,
  family = ig_family,
  prior = ig_prior
)

fit_linear <- fit_or_read_brms(
  file = file.path(resources_path, sprintf("fit_linear_ig_n%03d_sim%03d.rds", sample_size, sim_id)),
  formula = formula_linear,
  data = dat_ig,
  seed = 20260507 + sim_id - 1,
  family = ig_family,
  prior = ig_prior
)

```

6 Holdout Fits for HPC

The PPC checks use the full-data fits. The HPC checks use the same representative dataset split into a training subset and a held-out subset. The models are refitted on the training subset and evaluated on the held-out subset.

```

hpc_split_file <- file.path(resources_path, sprintf("hpc_split_n%03d_sim%03d.rds", sample_size, sim_id))

if (!force_rerun && file.exists(hpc_split_file)) {
  hpc_split <- readRDS(hpc_split_file)
  dat_ig_train <- hpc_split$train
  dat_ig_test  <- hpc_split$test
} else {
  set.seed(20260512 + sim_id - 1)
  n_test <- max(10L, floor(0.30 * nrow(dat_ig)))
  test_id <- sort(sample(seq_len(nrow(dat_ig)), size = n_test))
  dat_ig_train <- dat_ig[-test_id, , drop = FALSE]
  dat_ig_test  <- dat_ig[ test_id, , drop = FALSE]
}

```

```

saveRDS(list(train = dat_ig_train, test = dat_ig_test), hpc_split_file)
}

fit_piecewise_hpc <- fit_or_read_brms(
  file = file.path(resources_path, sprintf("fit_piecewise_ig_hpc_n%03d_sim%03d.rds", sample_size, sim_id)),
  formula = formula_piecewise,
  data = dat_ig_train,
  seed = 20260508 + sim_id - 1,
  family = ig_family,
  prior = ig_prior
)

fit_linear_hpc <- fit_or_read_brms(
  file = file.path(resources_path, sprintf("fit_linear_ig_hpc_n%03d_sim%03d.rds", sample_size, sim_id)),
  formula = formula_linear,
  data = dat_ig_train,
  seed = 20260509 + sim_id - 1,
  family = ig_family,
  prior = ig_prior
)

```

7 Compute Posterior and ISCV RSP Z-residuals

For continuous responses, the RSP is the ordinary survival probability, so no auxiliary randomization is needed. Therefore `nrep = 1` is sufficient.

```

force_recompute_zres_now <- if (exists("force_recompute_zres")) {
  isTRUE(force_recompute_zres)
} else {
  isTRUE(force_rerun)
}

zres_post_file <- file.path(
  resources_path,
  sprintf("zres_post_list_all_n%03d_sim%03d.rds", sample_size, sim_id)
)

zres_iscv_file <- file.path(

```

```

resources_path,
sprintf("zres_iscv_list_all_n%03d_sim%03d.rds", sample_size, sim_id)
)

required_zres_names <- c("Piecewise_IG", "Linear_IG")

if (
  !force_recompute_zres_now &&
  file.exists(zres_post_file) &&
  file.exists(zres_iscv_file)
) {
  zres_post_list <- readRDS(zres_post_file)
  zres_iscv_list <- readRDS(zres_iscv_file)

  missing_post <- setdiff(required_zres_names, names(zres_post_list))
  missing_iscv <- setdiff(required_zres_names, names(zres_iscv_list))

  if (length(missing_post) || length(missing_iscv)) {
    stop(
      "Saved Z-residual files do not contain the expected model names.\n",
      "Missing from posterior list: ", paste(missing_post, collapse = ", "), "\n",
      "Missing from ISCV list: ", paste(missing_iscv, collapse = ", "),
      call. = FALSE
    )
  }
} else {
  z_piecewise_post <- Zresidual::Zresidual(
    fit_piecewise,
    data = dat_ig,
    mcmc_summarize = "post",
    nrep = 1
  )

  z_linear_post <- Zresidual::Zresidual(
    fit_linear,
    data = dat_ig,
    mcmc_summarize = "post",
    nrep = 1
  )
}

```

```

)

z_piecewise_iscv <- Zresidual::Zresidual(
  fit_piecewise,
  data = dat_ig,
  mcmc_summarize = "iscv",
  nrep = 1
)

z_linear_iscv <- Zresidual::Zresidual(
  fit_linear,
  data = dat_ig,
  mcmc_summarize = "iscv",
  nrep = 1
)

zres_post_list <- list(
  Piecewise_IG = z_piecewise_post,
  Linear_IG = z_linear_post
)

zres_iscv_list <- list(
  Piecewise_IG = z_piecewise_iscv,
  Linear_IG = z_linear_iscv
)

saveRDS(zres_post_list, zres_post_file)
saveRDS(zres_iscv_list, zres_iscv_file)
}

z_piecewise_post <- zres_post_list$Piecewise_IG
z_linear_post <- zres_post_list$Linear_IG

z_piecewise_iscv <- zres_iscv_list$Piecewise_IG
z_linear_iscv <- zres_iscv_list$Linear_IG

zcov_piecewise <- Zresidual::Zcov(fit_piecewise, data = dat_ig)
zcov_linear <- Zresidual::Zcov(fit_linear, data = dat_ig)

```

8 Posterior Z-residual Diagnostic Plots

The plots below illustrate the main diagnostic idea. Under an adequate mean structure, the Z-residuals should be randomly scattered around zero. When the functional form for x_1 is too restrictive, the residual-versus- x_1 plot can reveal a systematic pattern even if an overall normality check is not strongly significant.

```
op <- par(no.readonly = TRUE)
par(mfrow = c(1, 4), mar = c(4, 4, 3, 1), oma = c(0, 0, 1, 0), cex = 1.05)

plot(
  z_piecewise_post,
  info = zcov_piecewise,
  x_axis_var = "x1",
  irep = 1,
  add_lowess = TRUE,
  main.title = expression("(a) IG Piecewise, " * Z^"Post-RSP" * " vs " * X[1])
)

plot(
  z_piecewise_post,
  info = zcov_piecewise,
  x_axis_var = "lp",
  irep = 1,
  add_lowess = TRUE,
  main.title = expression("(b) IG Piecewise, " * Z^"Post-RSP" * " vs LP")
)

plot(
  z_linear_post,
  info = zcov_linear,
  x_axis_var = "x1",
  irep = 1,
  add_lowess = TRUE,
  main.title = expression("(c) IG Linear, " * Z^"Post-RSP" * " vs " * X[1])
)

plot(
  z_linear_post,
  info = zcov_linear,
```

```

x_axis_var = "lp",
irep = 1,
add_lowess = TRUE,
main.title = expression("(d) IG Linear, " * ZPost-RSP * " vs LP")
)

par(op)

```

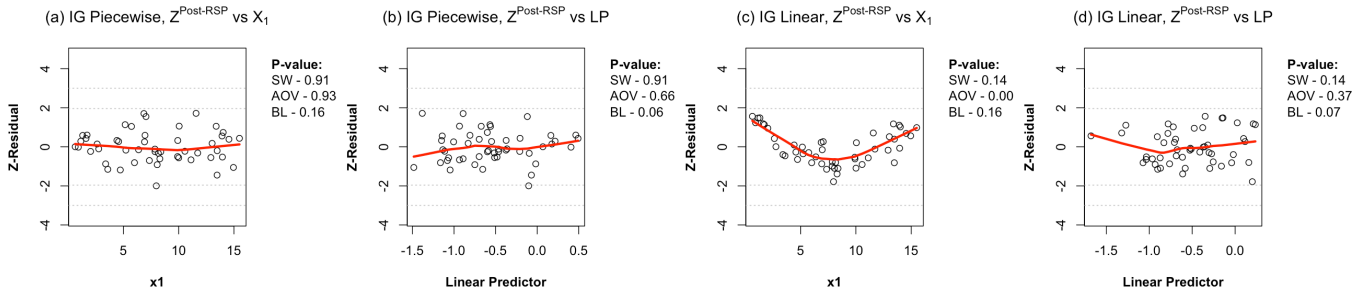


Figure 1: Visual diagnostics of posterior RSP Z-residuals for one continuous IG simulation dataset with $n = 50$. The piecewise IG model is correctly specified; the linear IG model is misspecified by replacing the piecewise quadratic effect of x_1 with a single linear term.

9 Compute PPC and HPC Checks

For comparison, we also compute PPC and HPC checks based on analogous statistics. This includes the traditional $y\text{-}\chi^2$ check and parameter-wise Z-residual PPC/HPC checks based on the SW and AOV statistics.

```

ndraws_check <- 2000
seed_check <- 20260511 + sim_id - 1
x_check <- c("lp", "x1")

force_recompute_ppc_hpc_now <- if (exists("force_recompute_ppc_hpc")) {
  isTRUE(force_recompute_ppc_hpc)
} else {
  isTRUE(force_rerun)
}

ppc_file <- file.path(
  resources_path,
  sprintf("ppc_list_all_n%03d_sim%03d.rds", sample_size, sim_id)
)

```

```

)

hpc_file <- file.path(
  resources_path,
  sprintf("hpc_list_all_n%03d_sim%03d.rds", sample_size, sim_id)
)

required_check_names <- c("Piecewise_IG", "Linear_IG")

if (
  !force_recompute_ppc_hpc_now &&
  file.exists(ppc_file) &&
  file.exists(hpc_file)
) {
  ppc_list <- readRDS(ppc_file)
  hpc_list <- readRDS(hpc_file)

  missing_ppc <- setdiff(required_check_names, names(ppc_list))
  missing_hpc <- setdiff(required_check_names, names(hpc_list))

  if (length(missing_ppc) || length(missing_hpc)) {
    stop(
      "Saved PPC/HPC files do not contain the expected model names.\n",
      "Missing from PPC list: ", paste(missing_ppc, collapse = ", "), "\n",
      "Missing from HPC list: ", paste(missing_hpc, collapse = ", "),
      call. = FALSE
    )
  }
} else {
  ppc_list <- list(
    Piecewise_IG = safe_ppc(
      fit = fit_piecewise,
      data = dat_ig,
      x = x_check,
      ndraws = ndraws_check,
      seed = seed_check,
      randomized = TRUE
    ),

```



```

Linear_IG = safe_ppc(
  fit = fit_linear,
  data = dat_ig,
  x = x_check,
  ndraws = ndraws_check,
  seed = seed_check,
  randomized = TRUE
)
)

hpc_list <- list(
  Piecewise_IG = safe_hpc(
    fit = fit_piecewise_hpc,
    newdata = dat_ig_test,
    x = x_check,
    ndraws = ndraws_check,
    seed = seed_check,
    randomized = TRUE
  ),

  Linear_IG = safe_hpc(
    fit = fit_linear_hpc,
    newdata = dat_ig_test,
    x = x_check,
    ndraws = ndraws_check,
    seed = seed_check,
    randomized = TRUE
  )
)

saveRDS(ppc_list, ppc_file)
saveRDS(hpc_list, hpc_file)
}

ppc_piecewise <- ppc_list$Piecewise_IG
ppc_linear    <- ppc_list$Linear_IG

hpc_piecewise <- hpc_list$Piecewise_IG

```

```
hpc_linear <- hpc_list$Linear_IG
```

10 Diagnostic Results for the Representative Dataset

The table reports posterior Z-residual tests, ISCV Z-residual tests, PPC tests, and HPC tests side by side for the representative dataset. Because this is a continuous response example, the reported values are ordinary p-values rather than replicated or median upper-bound p-values.

```
ig_result_table <- tibble::tibble(  
  Model = c("Piecewise IG", "Linear IG"),  
  
  Post_Z_SW = c(sw_pvalue(z_piecewise_post), sw_pvalue(z_linear_post)),  
  Post_Z_AOV_x1 = c(  
    aov_pvalue(z_piecewise_post, zcov_piecewise, "x1", k_anova = 10),  
    aov_pvalue(z_linear_post, zcov_linear, "x1", k_anova = 10)  
  ),  
  Post_Z_AOV_LP = c(  
    aov_pvalue(z_piecewise_post, zcov_piecewise, "lp", k_anova = 10),  
    aov_pvalue(z_linear_post, zcov_linear, "lp", k_anova = 10)  
  ),  
  Post_Z_BL_x1 = c(  
    bartlett_pvalue(z_piecewise_post, zcov_piecewise, "x1", k_anova = 10),  
    bartlett_pvalue(z_linear_post, zcov_linear, "x1", k_anova = 10)  
  ),  
  Post_Z_chisq = c(chisq_z_pvalue(z_piecewise_post), chisq_z_pvalue(z_linear_post)),  
  
  ISCV_Z_SW = c(sw_pvalue(z_piecewise_iscv), sw_pvalue(z_linear_iscv)),  
  ISCV_Z_AOV_x1 = c(  
    aov_pvalue(z_piecewise_iscv, zcov_piecewise, "x1", k_anova = 10),  
    aov_pvalue(z_linear_iscv, zcov_linear, "x1", k_anova = 10)  
  ),  
  ISCV_Z_AOV_LP = c(  
    aov_pvalue(z_piecewise_iscv, zcov_piecewise, "lp", k_anova = 10),  
    aov_pvalue(z_linear_iscv, zcov_linear, "lp", k_anova = 10)  
  ),  
  ISCV_Z_BL_x1 = c(  
    bartlett_pvalue(z_piecewise_iscv, zcov_piecewise, "x1", k_anova = 10),  
    bartlett_pvalue(z_linear_iscv, zcov_linear, "x1", k_anova = 10)
```

```

),
ISCV_Z_chisq = c(chisq_z_pvalue(z_piecewise_iscv), chisq_z_pvalue(z_linear_iscv)),

PPC_Z_SW = c(predcheck_scalar(ppc_piecewise, "sw_rpp"), predcheck_scalar(ppc_linear, "sw_rpp")),
PPC_Z_AOV_x1 = c(predcheck_aov(ppc_piecewise, "x1"), predcheck_aov(ppc_linear, "x1")),
PPC_Z_AOV_LP = c(predcheck_aov(ppc_piecewise, "lp"), predcheck_aov(ppc_linear, "lp")),
PPC_y_chisq = c(predcheck_scalar(ppc_piecewise, "chisq"), predcheck_scalar(ppc_linear, "chisq")),

HPC_Z_SW = c(predcheck_scalar(hpc_piecewise, "sw_rpp"), predcheck_scalar(hpc_linear, "sw_rpp")),
HPC_Z_AOV_x1 = c(predcheck_aov(hpc_piecewise, "x1"), predcheck_aov(hpc_linear, "x1")),
HPC_Z_AOV_LP = c(predcheck_aov(hpc_piecewise, "lp"), predcheck_aov(hpc_linear, "lp")),
HPC_y_chisq = c(predcheck_scalar(hpc_piecewise, "chisq"), predcheck_scalar(hpc_linear, "chisq")),
)

ig_result_table %>%
  gt::gt() %>%
  gt::tab_spanner(
    label = "Posterior Z-residual tests",
    columns = c(Post_Z_SW, Post_Z_AOV_x1, Post_Z_AOV_LP, Post_Z_BL_x1, Post_Z_chisq)
  ) %>%
  gt::tab_spanner(
    label = "ISCV Z-residual tests",
    columns = c(ISCV_Z_SW, ISCV_Z_AOV_x1, ISCV_Z_AOV_LP, ISCV_Z_BL_x1, ISCV_Z_chisq)
  ) %>%
  gt::tab_spanner(
    label = "PPC tests",
    columns = c(PPC_Z_SW, PPC_Z_AOV_x1, PPC_Z_AOV_LP, PPC_y_chisq)
  ) %>%
  gt::tab_spanner(
    label = "HPC tests",
    columns = c(HPC_Z_SW, HPC_Z_AOV_x1, HPC_Z_AOV_LP, HPC_y_chisq)
  ) %>%
  gt::cols_label(
    Model = "Model",
    Post_Z_SW = "Z-SW",
    Post_Z_AOV_x1 = "Z-AOV: x1",
    Post_Z_AOV_LP = "Z-AOV: LP",
    Post_Z_BL_x1 = "Z-BL: x1",
    Post_Z_chisq = gt::md("Z- $\chi^2$ "),

```

Representative IG simulation: diagnostic p-values

Model	Posterior Z-residual tests					ISCV Z-residual tests		
	Z-SW	Z-AOV: x1	Z-AOV: LP	Z-BL: x1	Z- χ^2	Z-SW	Z-AOV: x1	Z-AOV: LP
Piecewise IG	0.907	0.928	0.663	0.162	0.984	0.898	0.956	0.735
Linear IG	0.143	0.000	0.372	0.162	0.929	0.086	0.000	0.350

```

ISCV_Z_SW = "Z-SW",
ISCV_Z_AOV_x1 = "Z-AOV: x1",
ISCV_Z_AOV_LP = "Z-AOV: LP",
ISCV_Z_BL_x1 = "Z-BL: x1",
ISCV_Z_chisq = gt::md("Z- $\chi^2$ "),
PPC_Z_SW = "Z-SW",
PPC_Z_AOV_x1 = "Z-AOV: x1",
PPC_Z_AOV_LP = "Z-AOV: LP",
PPC_y_chisq = gt::md("$y$- $\chi^2$ "),
HPC_Z_SW = "Z-SW",
HPC_Z_AOV_x1 = "Z-AOV: x1",
HPC_Z_AOV_LP = "Z-AOV: LP",
HPC_y_chisq = gt::md("$y$- $\chi^2$ ")
) %>%
gt::fmt_number(columns = -Model, decimals = 3) %>%
gt::cols_align(align = "center", columns = dplyr::everything()) %>%
gt::cols_align(align = "left", columns = Model) %>%
gt::cols_width(Model ~ gt::px(110)) %>%
gt::tab_header(title = "Representative IG simulation: diagnostic p-values")

```

The most relevant diagnostic for the simpler linear model is the targeted AOV test against x_1 , because the potential misspecification is specifically in the functional form of $X_{i,1}$. Overall checks such as SW or χ^2 summarize marginal residual behavior and can be less sensitive to a localized residual pattern against a covariate.

11 Optional Full Replication Driver

The chunk below is not evaluated when the vignette is rendered. It shows how the same single-dataset workflow can be extended to a larger simulation with multiple replications and multiple sample sizes. It saves per-dataset p-values for posterior/ISCV Z-residual tests as well as PPC and HPC checks, so that p-value histograms and rejection-rate summaries can be created later if desired.

```

run_one_piecewise_ig <- function(
  n_sample,
  sim_id,
  base_seed = 20260505,
  ndraws_check = 2000,
  hpc_test_prop = 0.30
) {
  dat_file <- file.path(resources_path, sprintf("full_data_n%03d_sim%03d.rds", n_sample, sim_id))
  split_file <- file.path(resources_path, sprintf("full_hpc_split_n%03d_sim%03d.rds", n_sample, sim_id))

  fit_piece_file <- file.path(resources_path, sprintf("full_fit_piecewise_n%03d_sim%03d.rds", n_sample, sim_id))
  fit_linear_file <- file.path(resources_path, sprintf("full_fit_linear_n%03d_sim%03d.rds", n_sample, sim_id))
  fit_piece_hpc_file <- file.path(resources_path, sprintf("full_fit_piecewise_hpc_n%03d_sim%03d.rds", n_sample, sim_id))
  fit_linear_hpc_file <- file.path(resources_path, sprintf("full_fit_linear_hpc_n%03d_sim%03d.rds", n_sample, sim_id))

  if (!force_rerun && file.exists(dat_file)) {
    dat <- readRDS(dat_file)
  } else {
    dat <- sim_piecewise_ig(
      sample_size = n_sample,
      shape = 25,
      seed = base_seed + 10000 * n_sample + sim_id
    )
    saveRDS(dat, dat_file)
  }

  if (!force_rerun && file.exists(split_file)) {
    split <- readRDS(split_file)
    dat_train <- split$train
    dat_test <- split$test
  } else {
    set.seed(base_seed + 11000 * n_sample + sim_id)
    n_test <- max(10L, floor(hpc_test_prop * nrow(dat)))
    test_id <- sort(sample(seq_len(nrow(dat)), size = n_test))
    dat_train <- dat[-test_id, , drop = FALSE]
    dat_test <- dat[test_id, , drop = FALSE]
    saveRDS(list(train = dat_train, test = dat_test), split_file)
  }
}

```

```

fit_piece <- fit_or_read_brms(
  file = fit_piece_file,
  formula = formula_pieewise,
  data = dat,
  seed = base_seed + 20000 * n_sample + sim_id,
  family = ig_family,
  prior = ig_prior
)

fit_lin <- fit_or_read_brms(
  file = fit_linear_file,
  formula = formula_linear,
  data = dat,
  seed = base_seed + 30000 * n_sample + sim_id,
  family = ig_family,
  prior = ig_prior
)

fit_piece_hpc <- fit_or_read_brms(
  file = fit_piece_hpc_file,
  formula = formula_pieewise,
  data = dat_train,
  seed = base_seed + 40000 * n_sample + sim_id,
  family = ig_family,
  prior = ig_prior
)

fit_lin_hpc <- fit_or_read_brms(
  file = fit_linear_hpc_file,
  formula = formula_linear,
  data = dat_train,
  seed = base_seed + 50000 * n_sample + sim_id,
  family = ig_family,
  prior = ig_prior
)

z_piece_post <- Zresidual::Zresidual(fit_piece, data = dat, mcmc_summarize = "post", nrep = 1)
z_lin_post <- Zresidual::Zresidual(fit_lin, data = dat, mcmc_summarize = "post", nrep = 1)
z_piece_is <- Zresidual::Zresidual(fit_piece, data = dat, mcmc_summarize = "iscv", nrep = 1)

```

```

z_lin_is      <- Zresidual::Zresidual(fit_lin,  data = dat, mcmc_summarize = "iscv", nrep = 1)

zcov_piece <- Zresidual::Zcov(fit_piece, data = dat)
zcov_lin   <- Zresidual::Zcov(fit_lin,  data = dat)

ppc_piece <- safe_ppc(
  fit = fit_piece,
  data = dat,
  x = c("lp", "x1"),
  ndraws = ndraws_check,
  seed = base_seed + 60000 * n_sample + sim_id,
  randomized = TRUE
)

ppc_lin <- safe_ppc(
  fit = fit_lin,
  data = dat,
  x = c("lp", "x1"),
  ndraws = ndraws_check,
  seed = base_seed + 60000 * n_sample + sim_id,
  randomized = TRUE
)

hpc_piece <- safe_hpc(
  fit = fit_piece_hpc,
  newdata = dat_test,
  x = c("lp", "x1"),
  ndraws = ndraws_check,
  seed = base_seed + 70000 * n_sample + sim_id,
  randomized = TRUE
)

hpc_lin <- safe_hpc(
  fit = fit_lin_hpc,
  newdata = dat_test,
  x = c("lp", "x1"),
  ndraws = ndraws_check,
  seed = base_seed + 70000 * n_sample + sim_id,
  randomized = TRUE
)

```

```

)

dplyr::bind_rows(
  tibble::tibble(
    n = n_sample,
    sim_id = sim_id,
    model = "piecewise",
    method = c(
      "Post-RSP", "Post-RSP", "Post-RSP", "Post-RSP",
      "ISCV-RSP", "ISCV-RSP", "ISCV-RSP", "ISCV-RSP",
      "PPC", "PPC", "PPC", "PPC",
      "HPC", "HPC", "HPC", "HPC"
    ),
    test = c(
      "SW", "AOV_x1", "AOV_LP", "Z_chisq",
      "SW", "AOV_x1", "AOV_LP", "Z_chisq",
      "SW", "AOV_x1", "AOV_LP", "y_chisq",
      "SW", "AOV_x1", "AOV_LP", "y_chisq"
    ),
    p_value = c(
      sw_pvalue(z_piece_post),
      aov_pvalue(z_piece_post, zcov_piece, "x1", k_anova = 10),
      aov_pvalue(z_piece_post, zcov_piece, "lp", k_anova = 10),
      chisq_z_pvalue(z_piece_post),
      sw_pvalue(z_piece_is),
      aov_pvalue(z_piece_is, zcov_piece, "x1", k_anova = 10),
      aov_pvalue(z_piece_is, zcov_piece, "lp", k_anova = 10),
      chisq_z_pvalue(z_piece_is),
      predcheck_scalar(ppc_piece, "sw_rpp"),
      predcheck_aov(ppc_piece, "x1"),
      predcheck_aov(ppc_piece, "lp"),
      predcheck_scalar(ppc_piece, "chisq"),
      predcheck_scalar(hpc_piece, "sw_rpp"),
      predcheck_aov(hpc_piece, "x1"),
      predcheck_aov(hpc_piece, "lp"),
      predcheck_scalar(hpc_piece, "chisq")
    )
  ),
  tibble::tibble(

```



```

n = n_sample,
sim_id = sim_id,
model = "linear",
method = c(
  "Post-RSP", "Post-RSP", "Post-RSP", "Post-RSP",
  "ISCV-RSP", "ISCV-RSP", "ISCV-RSP", "ISCV-RSP",
  "PPC", "PPC", "PPC", "PPC",
  "HPC", "HPC", "HPC", "HPC"
),
test = c(
  "SW", "AOV_x1", "AOV_LP", "Z_chisq",
  "SW", "AOV_x1", "AOV_LP", "Z_chisq",
  "SW", "AOV_x1", "AOV_LP", "y_chisq",
  "SW", "AOV_x1", "AOV_LP", "y_chisq"
),
p_value = c(
  sw_pvalue(z_lin_post),
  aov_pvalue(z_lin_post, zcov_lin, "x1", k_anova = 10),
  aov_pvalue(z_lin_post, zcov_lin, "lp", k_anova = 10),
  chisq_z_pvalue(z_lin_post),
  sw_pvalue(z_lin_is),
  aov_pvalue(z_lin_is, zcov_lin, "x1", k_anova = 10),
  aov_pvalue(z_lin_is, zcov_lin, "lp", k_anova = 10),
  chisq_z_pvalue(z_lin_is),
  predcheck_scalar(ppc_lin, "sw_rpp"),
  predcheck_aov(ppc_lin, "x1"),
  predcheck_aov(ppc_lin, "lp"),
  predcheck_scalar(ppc_lin, "chisq"),
  predcheck_scalar(hpc_lin, "sw_rpp"),
  predcheck_aov(hpc_lin, "x1"),
  predcheck_aov(hpc_lin, "lp"),
  predcheck_scalar(hpc_lin, "chisq")
)
)
)
}

n_grid <- c(50, 100, 200)
n_rep <- 500

```

```

replication_results <- list()
idx <- 1L

for (n_now in n_grid) {
  for (sim_now in seq_len(n_rep)) {
    message("Running n = ", n_now, ", simulation ", sim_now)
    replication_results[[idx]] <- run_one_piecewise_ig(
      n_sample = n_now,
      sim_id = sim_now
    )
    idx <- idx + 1L
  }
}

replication_results <- dplyr::bind_rows(replication_results)
saveRDS(replication_results, file.path(resources_path, "section32_ig_replication_pvalues.rds"))

```

12 Practical Notes

- The rendered sections show the data-generating mechanism, fitted model formulas, representative posterior Z-residual plots, and single-dataset diagnostic p-values for a continuous IG example.
- The response is continuous, so the RSP residuals do not require repeated auxiliary randomization.
- Large multi-replication simulations can be run using the non-evaluated driver above, but they are not rendered by default because they require many repeated MCMC fits.